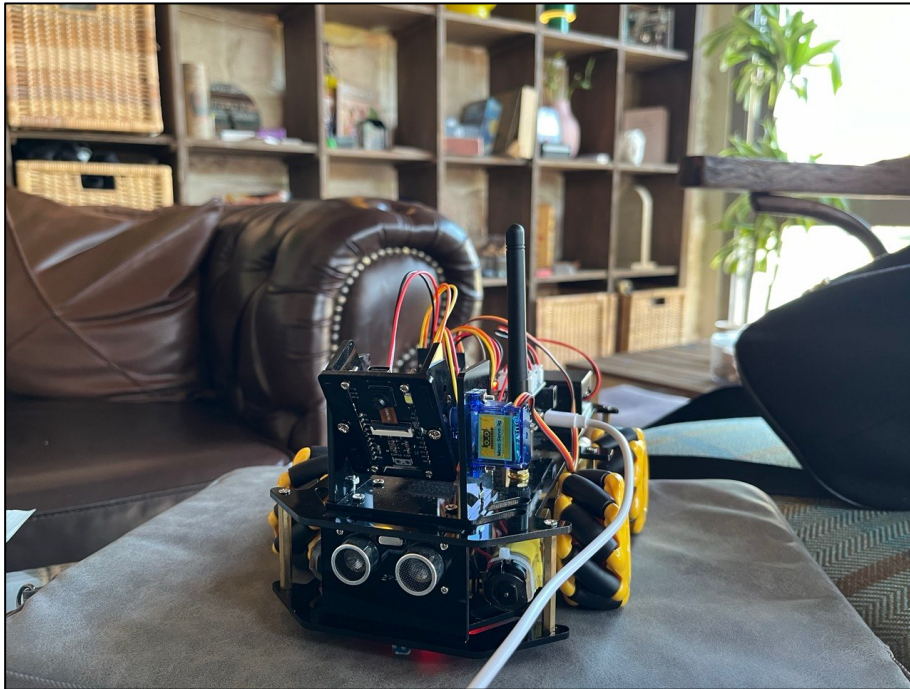


AI371 Course Project-F03

Cognitive Robotics



Supervised By Dr. Shurook Almohamade

N	SID	Student name
3	4456055	Shatha Ibrahim Ghabban
4	4451941	Shumukh Eid Alotaibi
5	4451274	Layal Mohammed Alhazmi
6	4454147	Hatoon Mohammed Ghabban

Keywords: AI, ML, Cognitive robotics, Mediapipe, Computer vision, Arduino

May 2026

Acknowledgement

We would like to express our sincere appreciation to Dr. Shurook Almohamade for her continuous encouragement, guidance, and valuable support throughout this project. Her enthusiasm for robotics and real-world applications inspired us to pursue a practical hardware-based implementation rather than limiting the project to simulation. Her insights and motivation played a significant role in shaping and improving our work.

We would also like to extend our gratitude to the Robotics Club in Madinah for their technical support and assistance during the hardware implementation and system integration phases of the project. Their efforts and collaboration helped us overcome several technical challenges throughout the development process.

Special thanks to Abdulaziz Albayoudh, a Computer Engineering student at King Saud University, for his valuable guidance and technical insights, which contributed significantly to the progress of this project.

Finally, we would like to thank everyone who supported us, directly or indirectly, and contributed to the successful completion of this work.



Table of Contents

Acknowledgement	2
List Of Figures	3
Abstract	4
1-Introduction & Background	5
2-Cognitive Model	6
2.1 Learning	6
2.2 perception	6
2.3 action	6
3. Architecture Diagram	7
4. Code Snippets.....	8
4.1 Gesture Recognition Model Training and Prediction	8
4.2 Command Sending from Python to Arduino	8
4.3 Arduino Command Execution and Obstacle Detection	9
4.4 Robot Movement Mapping	9
5. Evaluation	10
5.1 Perception Performance	10
5.2 Adaptability and Robustness	10
5.3 Human–Robot Interaction (HRI)	10
6. Challenges and Limitations	11
7. Future Improvements	11
References.....	12

List Of Figures

FIGURE1 :ARCHITECTURE COMPONENTS DIAGRAM	7
FIGURE2 :SYSTEM FLOW	7
FIGURE3 :SYSTEM ARCHITECTURE	7
FIGURE 4:CONFUSION MATRIX	10
FIGURE 5:CLASSIFICATION REPORT	10

Abstract

In this project, we designed and implemented an intelligent mobile robot capable of recognizing hand gestures through a live camera feed. The robot interprets gesture-based commands and responds in real time to control its movement direction. The system integrates computer vision and machine learning techniques to enable perception, decision-making, and immediate response. The trained gesture recognition model achieved approximately 98% accuracy during testing; however, some gestures with similar hand patterns occasionally caused misclassification. Overall, the project demonstrates the feasibility of integrating gesture recognition and cognitive robotics concepts into an interactive robotic system.

1-Introduction & Background

In this era of technological advancement, robots represent an important field in applying technology to reduce human effort, time, and costs. Therefore, robots have become valuable tools in many areas today. Based on this idea, this project utilizes key fields that help robots understand and interact with their environment, such as artificial intelligence, cognitive science, and computer vision.

The idea behind this project is to make the mobile robot move based on hand gestures and finger signals detected through live camera feed. Using supervised learning the robot recognizes the gesture and executes the corresponding command in real time, demonstrating perception and decision-making through vision.

Based on supervised learning and instructions, the robot moves, chooses, and executes the command, thus applying its perception and awareness of the world through vision.

Implementing such a project can contribute to improving efficiency and automation by providing continuous operation with reduced human effort and lower error rates.

2-Cognitive Model

The idea behind this project is to make the mobile robot move based on the number of fingers^[1] and their signals, as seen through a camera.

The robot performs key tasks centered around the following:

2.1 Learning

The robot learns by using hand landmark points extracted in real time through the camera. These landmark features are collected and used to train a supervised machine learning model to classify hand gesture commands.

2.2 perception

The robot perceives the signal through the camera, thus applying its awareness of the environment and interpreting the type of command to apply its perception^[2], and then converting this command into an action it takes.

2.3 action

The Mobile robot's response depends on the type of signal, which is interpreted as follows:

The signals are as follows:

5 fingers: Walk forward

1 finger: Go right

2 fingers: Go left

3 fingers: Go backward

0 fingers: Stop

In addition, the robot uses an ultrasonic sensor to detect obstacles during forward movement. When an obstacle is detected, the robot automatically stops to ensure safer navigation.

3. Architecture Diagram

The following diagrams illustrate the details and organization of the robot:

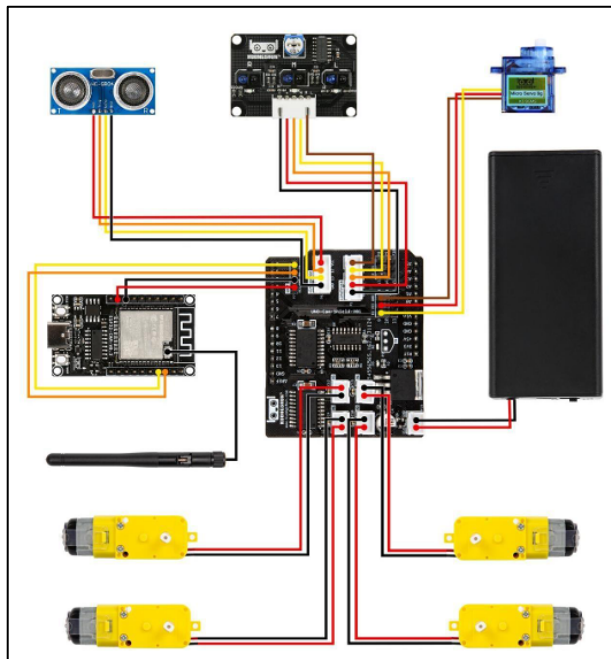


Figure 1 :Architecture Components Diagram

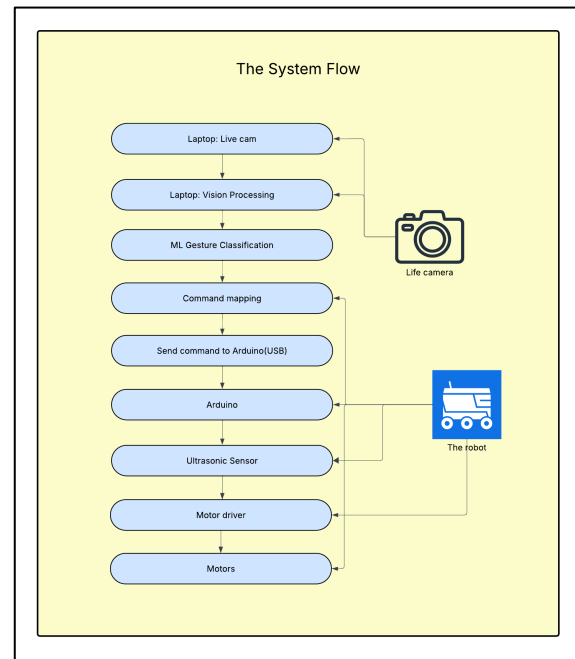


Figure 2 :System Flow

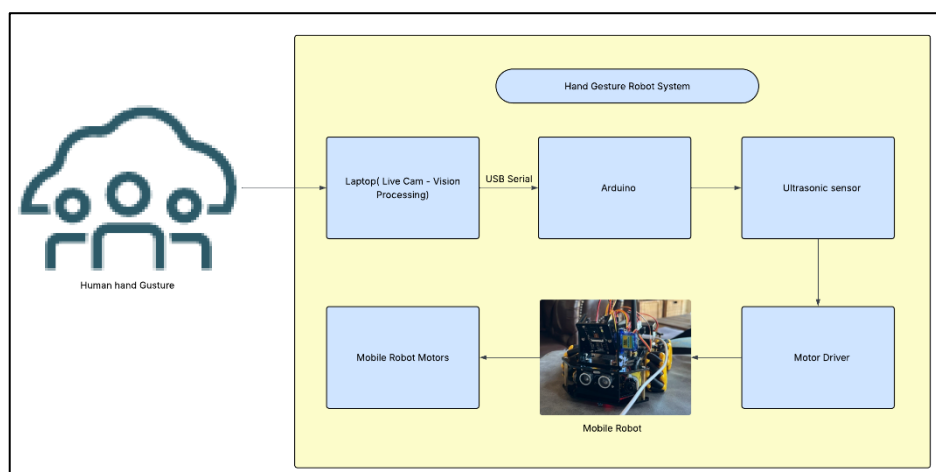


Figure 3 :System Architecture



4. Code Snippets

The code implementation was developed using references [3] [4] and [5].

4.1 Gesture Recognition Model Training and Prediction

```
# Train model
model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

model.fit(X_train, y_train)
# Save trained model
joblib.dump(model, model_file)
model_path = "gesture_model.pkl"
model = joblib.load(model_path)
X_input, finger_count = extract_features(hand_landmarks, hand_label)

try:
    predicted = model.predict(X_input)[0]
    print("Predicted:", predicted)

    if predicted in ["F", "B", "L", "R", "S"]:
        command = predicted
    else:
        command = "S"
```

4.2 Command Sending from Python to Arduino

```
SERIAL_PORT = "/dev/cu.usbserial-1110"
BAUD_RATE = 9600
ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
time.sleep(2)
last_sent_command = ""
last_send_time = 0
def send_command(command: str) -> None:
    global last_sent_command, last_send_time

    current_time = time.time()

    if command == last_sent_command and (current_time -
last_send_time) < 0.2:
        return

    try:
        ser.write((command + '\n').encode())
        print("Sent:", command)

        last_sent_command = command
        last_send_time = current_time

    except Exception as e:
        print("Serial error:", e)
```


4.3 Arduino Command Execution and Obstacle Detection

```
if (Serial.available() > 0) {  
    String cmd = Serial.readStringUntil('\n');  
    cmd.trim();  
  
    if (cmd.length() == 0) return;  
  
    char c = cmd.charAt(0);  
    float distance = getDistance();  
  
    Serial.print("Command: ");  
    Serial.print(c);  
    Serial.print(" | Distance: ");  
    Serial.print(distance);  
    Serial.println(" cm");  
  
    if (c == 'F' && distance < SAFE_DISTANCE) {  
        stopRobot();  
    }  
    else {  
        executeCommand(c);  
    }  
}
```

4.4 Robot Movement Mapping

```
void executeCommand(char c) {  
    switch (c) {  
        case 'F': forwardRobot(); break;  
        case 'B': backwardRobot(); break;  
        case 'L': leftRobot(); break;  
        case 'R': rightRobot(); break;  
        case 'S': stopRobot(); break;  
        default: stopRobot(); break;  
    }  
}
```

5. Evaluation

5.1 Perception Performance

The performance of the hand gesture recognition model was evaluated using a test dataset consisting of 152 samples. The system achieved an overall accuracy of approximately 98%, demonstrating stable and reliable performance in classifying robot movement commands.

To further evaluate the model, a classification report including precision, recall, and F1-score was generated for each gesture class. The results showed strong performance across all movement commands, with only minor confusion occurring between some similar gestures.

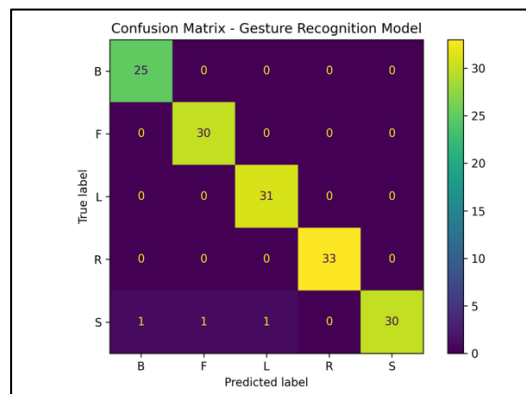


Figure 4:Confusion Matrix

Accuracy: 98.03%

Classification Report:

	precision	recall	f1-score	support
B	0.96	1.00	0.98	25
F	0.97	1.00	0.98	30
L	0.97	1.00	0.98	31
R	1.00	1.00	1.00	33
S	1.00	0.91	0.95	33
accuracy			0.98	152
macro avg	0.98	0.98	0.98	152
weighted avg	0.98	0.98	0.98	152

Figure 5:Classification Report

5.2 Adaptability and Robustness

The system was tested under different lighting conditions to evaluate its robustness and adaptability during real-world operation. The gesture recognition system was tested under normal indoor lighting and low-light environments. The system maintained stable performance under normal lighting conditions, while slight decreases in recognition accuracy were observed in low-light conditions due to reduced hand visibility.

5.3 Human–Robot Interaction (HRI)

The system was tested on multiple users with different hand sizes and gesture styles to evaluate the robot's responsiveness and ease of use. Most users were able to control the robot successfully using hand gestures in real time. The interaction between the user and the robot was smooth and responsive under normal indoor conditions. Some hand positions occasionally affect gesture recognition accuracy.

6. Challenges and Limitations

One of the main challenges faced in this project was the difficulty of integrating Python code with Arduino code via Wi-Fi using the ESP32-CAM [6] board. Despite multiple attempts and trying different approaches, a stable connection could not be achieved.

One of the key limitations of the project is that the communication between the Python code and the Arduino code relies on a wired connection, which restricts the robot's mobility and flexibility.

Additionally, the robot is unable to operate effectively on uneven surfaces due to the limitations of its wheeled design.

In some Cases, the gesture recognition model may become confused with similar hand gestures, which can affect the accuracy of command classification and the robot's response.

The current obstacle detection feature relies on a single front ultrasonic sensor, which limits obstacle detection coverage during robot navigation.

7. Future Improvements

In future work, there are several opportunities to enhance the system's performance and expand its capabilities. One potential improvement is enabling a wireless connection between the Python code and the robot to increase mobility and flexibility.

Another area for improvement is the integration of additional sensors, such as LiDAR (Light Detection and Ranging) technology This would provide the robot with high-precision, allowing for advanced obstacle avoidance and environmental mapping while maintaining real-time responsiveness to hand gestures.

Future work may focus on improving the gesture recognition system by using more advanced classification techniques and optimizing gesture mapping to achieve more stable and reliable robot responses.

References

- [1] M. Correa, J. Ruiz-del-Solar, R. Verschae and J. Lee-Ferng, "Real-Time Hand Gesture Recognition for Human Robot Interaction," 2009. [Online]. Available: Real-Time Hand Gesture Recognition for Human Robot Interaction.
- [2] M. Correa, J. Ruiz-del-Solar, R. Verschae, J. Lee-Ferng and N. Castillo, "Real-Time Hand Gesture Recognition for Human Robot".
- [3] C. Liechti, "Welcome to pySerial's documentation¶," 2001. [Online]. Available: <https://pythonhosted.org/pyserial/>.
- [4] D. Kleine, "Finger counter with OpenCV and MediaPipe," 2023. [Online]. Available: https://github.com/d-kleine/finger_counter_webcam/tree/main.
- [5] "ESP32-CAM Smart Robot Car Tutorial Manual. (n.d.). Tutorial manual provided with the purchased robot kit.," ielectroney. [Online].
- [6] M. F. A. C. Saputro, S. Hadiyoso and I. Irawati, "Real-Time Hand Gesture-Based Virtual Mouse System Using ESP32-CAM and OpenCV," February 2006. [Online]. Available: Based_Virtual_Mouse_System_Using_ESP32.